

E. 分蛋糕 4

Problem ID: Cake_4

Time Limit: 1s

Memory Limit: 512MiB

本題為互動題，需要實作兩個檔案。

南宮柳棲今天邀請了資訊社的大家參加他的派對。派對中有一個超大超大的蛋糕，從左到右編號為第 0 段至第 $n - 1$ 段。

為了追求公平，蛋糕必須被徹底切分，直到所有蛋糕都成為單獨的一小塊。對於任何一段連續的蛋糕區間 $[l, r]$ ，切蛋糕的人可以選擇一個位置 k ($l \leq k < r$) 切一刀，將其分成 $[l, k]$ 與 $[k + 1, r]$ 兩塊，接著再分別將左右兩塊徹底切分。

對每個區間 $[l, r]$ ，都有一個事先決定的非負整數 $w_{l,r}$ 。每當對區間 $[l, r]$ 切下第一刀時，柳棲想要跳起來大喊「6767」的衝動值會增加 $w_{l,r}$ 。單獨一小塊不需要再切，因此 $w_{i,i} = 0$ 。

舉例來說，如果選擇在位置 k 切開 $[l, r]$ ，這次完整切分所產生的總衝動值為 $w_{l,r} + \text{cost}(l, k) + \text{cost}(k + 1, r)$

其中 $\text{cost}(x, y)$ 代表將區間 $[x, y]$ 徹底切分所需的最小總衝動值。

這份費用表 w 雖然是事先決定的，但柳棲知道它完全反映了蛋糕的物理尺寸與視覺張力，並保證滿足兩個極為自然的規律：

- **大就是好**：蛋糕區間越長，視覺震撼越強。對於任何一段連續的蛋糕，如果只取其中一部分連續的子區間來切，該子區間產生的衝動值絕對不會超過原本整段大區間的衝動值。
- **越長越好**：蛋糕的視覺衝擊具有邊際遞增的效應。想像有兩段右邊界對齊、但左邊界不同的蛋糕區間（即一長一短）。如果將這兩段蛋糕的右邊界同時向右延伸一段相同的長度，那麼原本就比較長（左邊界較遠）的那段蛋糕，它所產生的「衝動值增加量」，一定大於等於較短蛋糕的衝動值增加量。

柳棲有著預知未來的能力，他知道完整的費用表 w 。但是，在派對上實際負責切蛋糕的巴漆並不知道任何 $w_{l,r}$ 。

柳棲只能在派對開始前，傳送一段長度不超過 B 個位元的二進位編碼給巴漆。派對開始後，現場會出現 Q 個詢問。每次詢問會給定一個區間 $[l, r]$ ，巴漆必須回答：**如果只拿出區間 $[l, r]$ 的蛋糕進行徹底切分，為了讓總衝動值最小，第一刀需要切在哪個位置 k ？**如果在多個位置切第一刀都能得到最小總衝動值，回答其中任意一個即可。

— 實作說明 —

本題是 **Two Step** 題型。你需要上傳兩個檔案。

柳棲

第一個檔案為 `sixseven.cpp`，你必須在該檔案內實作 `encode` 這個函式：

```
std::string encode(int n, std::vector<std::vector<long long> > w);
```

- n 是蛋糕的總段數。
- w 是一個 $n \times n$ 的二維陣列。
- 對所有 $0 \leq l \leq r < n$ ， $w[l][r]$ 等於 $w_{l,r}$ 。
- 當 $l > r$ 時， $w[l][r]$ 的內容沒有意義，請勿使用。
- 你必須回傳一個只包含字元 0 與 1、長度不超過 B 的字串。

巴漆

第二個檔案為 `eightseven.cpp`，你必須在該檔案內實作 `decode` 與 `query` 兩個函式：

```
void decode(int n, std::string S);
```

- n 是蛋糕的總段數。
- S 是第一個程式中 `encode` 回傳的二進位字串。
- 第二個程式不會取得費用表 w 。

```
int query(int l, int r);
```

- l 、 r 代表本次詢問的蛋糕區間 $[l, r]$ 。
- 你必須回傳一個最佳的第一刀切點 k ，且 $l \leq k < r$ 。
- 如果有多個最佳切點，回傳其中任意一個即可。

— 執行細節 —

- 你上傳的兩個檔案會與評測端準備的 `stub.cpp` 一同編譯成兩個獨立的執行檔，你可以在兩個檔案內宣告任何全域變數或內部函數，但請將這些函數宣告在匿名命名空間內以避免變數撞名，詳細請參考範例程式碼。
- 評測端會額外準備 `manager.cpp` 來操作兩個執行檔之間的互動。具體來說，評測端在編譯出三份執行檔 `manager`、`sixseven` 和 `eightseven` 後，三份執行檔將會同時運行，並用以下方式測試：
 1. `manager` 從輸入檔讀入輸入
 2. `manager` 將所需資料透過 `pipe` 傳給程式 `sixseven`，並呼叫 `encode()`
 3. `sixseven` 在收到 `encode()` 的回傳值後，將回傳值透過 `pipe` 傳給 `manager`
 4. `manager` 將所需資料透過 `pipe` 傳給程式 `eightseven`，並呼叫 `decode()`
 5. 接著 `manager` 將會執行 Q 次的詢問，每次呼叫 `query()`
 6. 每次詢問的時候，`eightseven` 收到 `query()` 的回傳值，將回傳值透過 `pipe` 回傳給 `manager` 並判斷結果
- 評分時 `sixseven` 和 `eightseven` 分別是兩個不同的執行緒，這代表兩分程式之間的全域變數或內部函數是不能共用的。你也不能使用任何方法來和任何外部檔案互動，唯輸出至 `stderr` 不在此限。
- `manager.cpp` 跟 `stub.cpp` 的所花時間微小至可忽略不計。

— 測資限制 —

- $2 \leq n \leq 2000$
- $0 \leq w_{l,r} \leq 3 \times 10^{16}$
- $w_{i,i} = 0$
- 費用表 w 滿足題目開頭所述的兩項性質
- $1 \leq Q \leq 10^5$
- $B = 4 \times 10^6$
- 每次詢問皆滿足 $0 \leq l < r < n$

— 互動範例 —

考慮附件中第一筆範例測試資料的情境

$$n = 4, Q = 3$$

$$w = \begin{pmatrix} 0 & 1 & 3 & 19 \\ & 0 & 2 & 18 \\ & & 0 & 16 \\ & & & 0 \end{pmatrix}.$$

一個可能被評為 Accepted 的互動例子顯示如下：

評分程式端	參賽者端
呼叫 <code>encode(4, W)</code>	
	回傳 S
呼叫 <code>decode(4, S)</code>	
呼叫 <code>query(0, 3)</code>	
	回傳 2
呼叫 <code>query(0, 1)</code>	
	回傳 0
呼叫 <code>query(1, 3)</code>	
	回傳 2

- W 的值為 $\{\{0, 1, 3, 19\}, \{87, 0, 2, 18\}, \{87, 87, 0, 16\}, \{87, 87, 87, 0\}\}$
- S 的值為 “100010101000010101010101111010100010” (我隨便亂打的)

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	27	$2 \leq n \leq 50, 1 \leq Q \leq 1000$
3	18	$2 \leq n \leq 200$
4	41	$2 \leq n \leq 800$
5	14	無額外限制

— 範例評分程式 —

為了測試方便，範例評分程式不完全按照執行細節段落內所講的執行，而是會將你的程式編譯成單一執行檔執行後，直接在該次執行依序呼叫 `sixseven` 和 `eightseven` 兩支程式。請注意，在範例評分程式執行的過程中，`sixseven` 與 `eightseven` 將可以共享不在匿名空間內的全域變數或內部函數，**但這在正式評分程式是不允許的**，因此還請特別注意別誤共享了資訊。

請使用 `compile_cpp.sh` 或 `compile_cpp.bat` 來編譯你寫好的兩支程式。並使用 `run_cpp.sh` 或 `run_cpp.sh` 執行範例評分程式，它將採用以下格式輸入：

```
n Q
w0,0, w0,1, ..., w0,n-1
w1,1, w1,2, ..., w1,n-1
⋮
wn-1,n-1
l1, r1
l2, r2
⋮
lQ, rQ
```

範例評分程式首先呼叫 `encode(n, w)`。接著，若範例評分程式偵測到 `encode()` 的回傳值有任何不合法，此程式將輸出

```
wrong answer: msg
```

後並終止程式執行，而 `msg` 為下列其中之一錯誤訊息。

- `Length limit exceeded` : `encode()` 的回傳字串長度超過 B 個字元。
- `Invalid returned character` : `encode()` 的回傳字串不是一個 01 字串。

否則，範例評分程式將會輸出一行

```
S
```

接著呼叫 `decode(n, S)`，其中的 S 將會是呼叫 `encode()` 所拿到的回傳字串。

接著範例評分程式會依序執行 Q 次詢問，每次呼叫 `query(l, r)`，並以以下格式印在標準輸出中：

```
i : query(l, r) k
```

- l, r 在第 i 次詢問中為 l_i, r_i
- k 為 `query(l, r)` 所得到的回傳值